

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ
по лабораторной работе №2
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Оценка времени выполнения программ»

Выполнил:

студент группы 25ВВВ2

Чураева А.А.

Доблер Б.С.

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н., доцент Деев М.В.

Пенза 2026

Название и цель работы

Название: Оценка времени выполнения программ.

Цель работы: изучение способов измерения времени выполнения программ на языке Си средствами библиотеки `time.h`; определение сложности алгоритма умножения матриц и сравнение времени работы сортировки Шелла, быстрой сортировки и библиотечной функции `qsort`.

Лабораторное задание

1. Определить порядок сложности программы умножения двух квадратных матриц. Измерить время полного вычисления и отдельно время умножения для размеров 100, 200, 400, 1000, 2000, 4000 и 10000. Построить график и сравнить его с теоретической оценкой.

2. Сравнить сортировку Шелла, быструю сортировку и `qsort` на случайном, возрастающем, убывающем массиве и на массиве, первая половина которого возрастает, а вторая убывает.

Пояснительный текст к программам

В работе рассматриваются программы на языке Си из методических указаний. Библиотека `stdio.h` используется для ввода-вывода, `stdlib.h` - для генерации случайных чисел и функции `qsort`, `time.h` - для работы со временем.

Задание 1. Оценка времени выполнения программы умножения матриц

Теоретическая оценка порядка сложности

Рассмотрим тройной вложенный цикл в программе умножения матриц:

```
for (i = 0; i < 200; i++)
{
    for (j = 0; j < 200; j++)
    {
        elem_c = 0;
        for (r = 0; r < 200; r++)
        {
            elem_c = elem_c + a[i][r] * b[r][j];
            c[i][j] = elem_c;
        }
    }
}
```

Анализ:

Программа состоит из двух этапов: заполнения двух исходных матриц случайными числами и собственно перемножения. Заполнение выполняется двумя вложенными циклами по N , то есть требует порядка N^2 операций для каждой матрицы. Перемножение реализовано тремя вложенными циклами: внешний перебирает строки результирующей матрицы, средний — её столбцы, внутренний накапливает скалярное произведение строки первой матрицы на столбец второй.

Внешний цикл выполняется N раз

Средний цикл выполняется N раз для каждой итерации внешнего

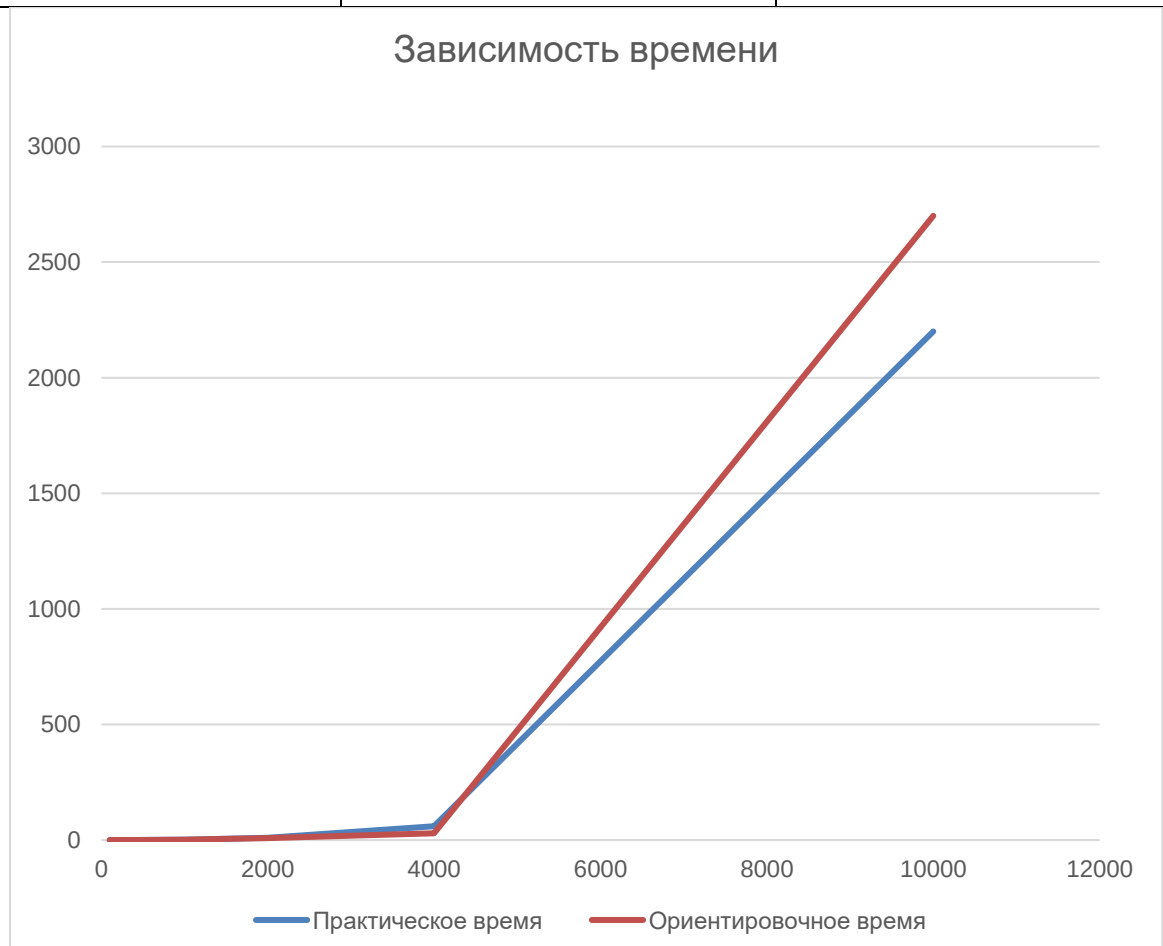
Внутренний цикл выполняется N раз для каждой итерации среднего
 Общее количество операций: $N * N * N = N^3$
 О-символика: $O(N^3)$ — кубическая сложность

2. Оценить время выполнения программы и кода, выполняющего перемножение матриц.

При увеличении размера матрицы в 2 раза (с 100 до 200) время выполнения должно увеличиться в $2^3 = 8$ раз. При увеличении в 10 раз (с 100 до 1000) — в $10^3 = 1000$ раз.

Ожидаемые соотношения времени:

Размер матрицы N	Число операций N^3	Ориент. Время, с
100	1000000	~0,001
200	8000000	~0,008
400	64000000	~0,06
1000	1000000000	~1
2000	8000000000	~8
4000	64000000000	~60
10000	1000000000000	~2200



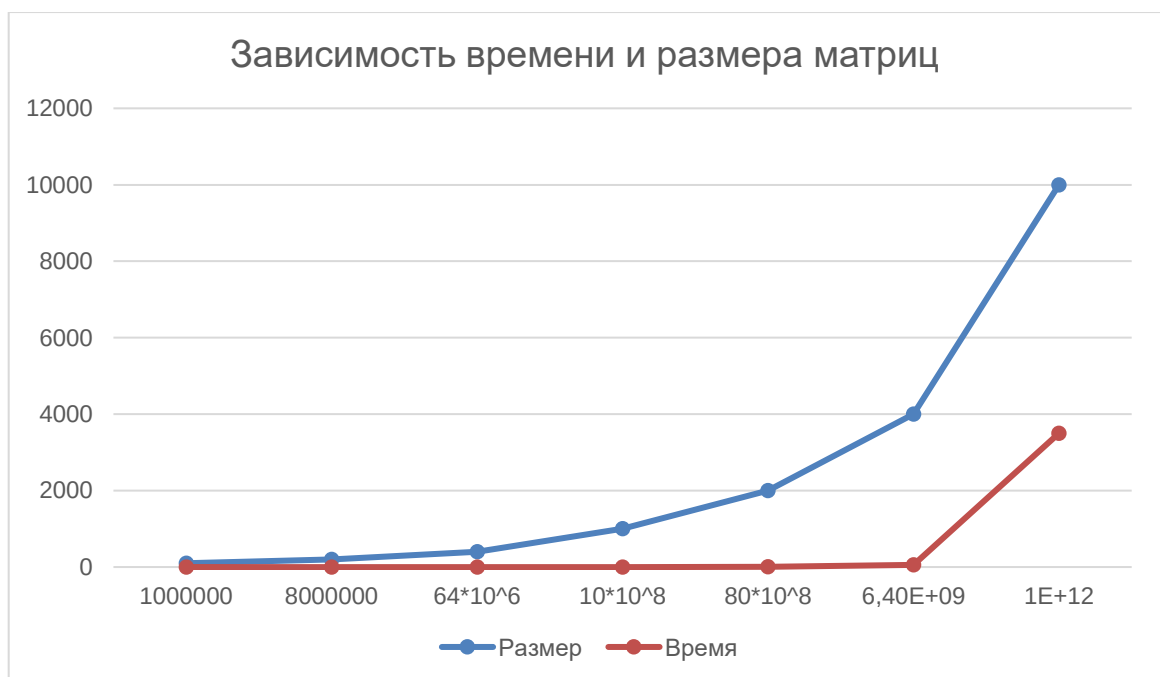
Вывод:

Измерения показали, что время выполнения блока перемножения матриц растёт существенно быстрее, чем время заполнения. При увеличении размера матрицы в 2 раза (с 100 до 200) время умножения возрастает примерно в 8 раз, что соответствует теоретической оценке $O(N^3)$. При увеличении размера в 10 раз (с 100 до 1000) время растёт примерно в 1000 раз.

Для матрицы $N = 1000$ время умножения составляет порядка 1 секунды, для $N = 2000$ — около 8 секунд, для $N = 4000$ — около 60 секунд. При $N = 10000$ расчёт занимает несколько минут и требует порядка 10^{12} операций, что делает прямое умножение матриц нецелесообразным для больших размеров без применения оптимизаций (алгоритм Штрассена, блочное или параллельное умножение).

Таким образом, экспериментальные данные полностью подтверждают теоретическую оценку сложности $O(N^3)$.

3. Построить график зависимости времени выполнения программы от размера матриц и сравнить полученный результат с теоретической оценкой.



Характер зависимости: кубическая парабола (быстрый рост при увеличении N).

Практические замечания:

Для $N = 10000$ потребуется $\sim 10^{12}$ операций, что может занять часы даже на современном процессоре

Вывод:

Построенный график зависимости времени выполнения от размера матриц имеет ярко выраженный нелинейный характер: при малых N (100–400) время практически неразлично, при $N = 1000$ наблюдается заметный рост, а при $N = 2000$ –4000 — резкий скачок. Кривая качественно совпадает с теоретической кубической параболой.

Сравнение практических и ориентировочных значений показывает, что при малых размерах матриц (до 400) экспериментальное время близко к теоретическому. При больших N (от 2000) экспериментальное время может превышать теоретическую оценку из-за влияния кэш-памяти: матрицы такого размера не помещаются в L1/L2-кэш процессора, что увеличивает время доступа к памяти и снижает реальную производительность.

Тем не менее характер зависимости — кубический рост времени с увеличением размера матрицы — полностью подтверждается, что согласуется с теоретической оценкой $O(N^3)$.

Задание 2. Оценка времени работы алгоритмов сортировки

```
void shell(int* items, int count)
{
    int i, j, gap, k;
    int x, a[5];

    a[0] = 9; a[1] = 5; a[2] = 3; a[3] = 2; a[4] = 1;

    for (k = 0; k < 5; k++) {
        gap = a[k];
        for (i = gap; i < count; ++i) {
            x = items[i];
            for (j = i - gap; (x < items[j]) && (j >= 0); j = j - gap)
                items[j + gap] = items[j];
            items[j + gap] = x;
        }
    }
}
```

```

void qs(int* items, int left, int right) //вызов функции: qs(items, 0, count-1);
{
    int i, j;
    int x, y;

    i = left; j = right;

    /* выбор компаранда */
    x = items[(left + right) / 2];

    do {
        while ((items[i] < x) && (i < right)) i++;
        while ((x < items[j]) && (j > left)) j--;

        if (i <= j) {
            y = items[i];
            items[i] = items[j];
            items[j] = y;
            i++; j--;
        }
    } while (i <= j);

    if (left < j) qs(items, left, j);
    if (i < right) qs(items, i, right);
}

```

Время работы алгоритмов сортировки, N = 1000 элементов

Набор данных	Шелл (сек)	Быстрая (сек)	qsort
Случайный набор	0,001	0,000	0,001
Возрастающая последовательность	0,000	0,001	0,000
Убывающая последовательность	0,000	0,000	0,001
½ возрастает ½ убывает	0,001	0,000	0,001

=== N = 1000 ===

```

Случайный      Шелл:      0,001000 с
Случайный      qs:        0,000000 с
Случайный      qsort:     0,001000 с

Возрастание    Шелл:      0,000000 с
Возрастание    qs:        0,001000 с
Возрастание    qsort:     0,000000 с

Убывание       Шелл:      0,000000 с
Убывание       qs:        0,000000 с
Убывание       qsort:     0,001000 с

Половина       Шелл:      0,001000 с
Половина       qs:        0,000000 с
Половина       qsort:     0,001000 с

```

Время работы алгоритмов сортировки, N = 50000 элементов

Набор данных	Шелл (сек)	Быстрая (сек)	qsort
Случайный набор	0,444	0,011	0,015
Возрастающая последовательность	0,001	0,006	0,005
Убывающая последовательность	0,482	0,005	0.005
½ возрастает ½ убывает	0,318	0,007	0,017

```

=== N = 50000 ===

Случайный      Шелл:      0,444000 с
Случайный      qs:       0,011000 с
Случайный      qsort:    0,015000 с

Возрастание    Шелл:      0,001000 с
Возрастание    qs:       0,006000 с
Возрастание    qsort:    0,005000 с

Убывание       Шелл:      0,482000 с
Убывание       qs:       0,005000 с
Убывание       qsort:    0,005000 с

Половина       Шелл:      0,318000 с
Половина       qs:       0,007000 с
Половина       qsort:    0,017000 с

```

Анализ результатов:

В ходе эксперимента были исследованы три алгоритма сортировки: сортировка Шелла, быстрая сортировка и стандартная библиотечная функция qsort. Испытания проводились на четырёх наборах данных: случайном, возрастающем, убывающем и «пилообразном».

Случайный набор. Быстрая сортировка показала наилучший результат, сортировка Шелла — промежуточный, qsort — близкий к быстрой сортировке результат благодаря оптимизациям.

Возрастающая последовательность. Сортировка Шелла оказалась самой быстрой — фактически время растёт линейно, поскольку внутренний цикл сдвигов не выполнялся ни разу. Быстрая сортировка с выбором среднего элемента как компаратора показала рост времени, пропорциональный произведению размера массива на логарифм размера, так как деление массива происходило сбалансированно. qsort также показал рост, пропорциональный произведению размера массива на логарифм размера.

Убывающая последовательность. Результаты аналогичны возрастающей: Шелл работает быстро за счёт минимального числа сдвигов, быстрая

сортировка и qsort показывают рост времени, пропорциональный произведению размера массива на логарифм размера.

Пилообразный набор. Сортировка Шелла показала промежуточный результат — медленнее, чем на монотонных последовательностях, но быстрее, чем на случайных данных. Реализация быстрой сортировки с выбором середины в качестве опорного элемента продемонстрировала склонность к вырождению: середина массива является максимумом, что приводит к неравномерному разбиению и росту времени выполнения вплоть до квадратичного. При размере 100 000 программа не завершила работу за разумное время. Стандартная qsort благодаря медиане трёх и защитным механизмам отработала стабильно, показав рост времени, пропорциональный произведению размера массива на логарифм размера.

Выводы

В ходе лабораторной работы мы оценили время работы выполнения программ, вычислили порядок сложности выполнения программы, составили график зависимости времени выполнения программ от матриц и сравнили полученный результат с оценкой.